Universidad de Costa Rica

Escuela de Ciencias de la Computación e Informática

Programación Paralela y Concurrente (CI-0117)

Informe Segundo Proyecto:

Servidor de pipelines concurrente

Docente:
Sleyter Ángulo

Estudiantes:
Santiago Asunción Marín - C5C829
Miguel Fabricio Arias Vargas - C5C762
Andrés Oreamuno Varela - A43842

II Semestre, 2026

1. Introducción

En el presente informe se describe el diseño, la implementación y las pruebas realizadas de un servidor de *pipelines* concurrente desarrollado en C. El servidor recibe las definiciones de flujos de trabajo escritas en archivos de formato *YAML*, las valida y posteriormente las ejecuta de forma paralela, haciendo uso de la biblioteca *OpenMP* para la ejecución de los trabajos y el procesamiento de datos. Adicionalmente, se hace uso de la biblioteca *Pthreads* para la capa de red en el servidor. Un *pipeline* se compone de etapas, trabajos y pasos, con dependencias entre ellos, las acciones que no tienen dependencias pueden ser ejecutadas al mismo tiempo. Esto genera que los *pipelines* compitan entre ellos por un número limitado de agentes.

En este proyecto se construye un programa compuesto por dos archivos ejecutables: `pipelined`, que es el servidor, y `pipectl`, el cliente.

El servidor escucha conexiones *TCP* y crea un hilo manejador por cliente, este interpreta una línea de texto con uno de los comando *SUBMIT*, *STATUS*, *LOGS*, *CANCEL*, *LIST* o *SHUTDOWN*

Mediante el comando *SUBMIT*, el usuario envía un *pipeline* para su ejecución. El servidor verifica que sea un archivo valido, en caso de encontrarse errores se muestra al usuario en que linea del archivo se encuentra el error. Si el archivo es valido, se le asigna un ID, con este ID el usuario puede consultar el estado y la duración de cada etapa, trabajo y paso con *STATUS*, ver la bitácora de ejecución mediante *LOGS*, incluso en tiempo real con la opción *FOLLOW*, y detener la ejecución con *CANCEL*. Además, *LIST* muestra todas las ejecuciones conocidas por el servidor, y *SHUTDOWN* realiza un apagado ordenado.

El servidor garantiza que nunca haya mas trabajos en ejecución que agentes disponibles, sin importar cuántos *pipelines* se envíen al mismo tiempo. Algunos trabajos usan tareas ya incluidas en el servidor, como contar las palabras de un archivo. Estas tareas reparten el trabajo entre varios hilos con *OpenMP*, y siempre dan el mismo resultado sin importar cuántos hilos se usen.

Para que el resultado de un *pipeline* no dependa del orden en que se ejecuten los hilos, se identifican los recursos compartidos y se protegen con mecanismos de sincronización, por ejemplo *mutex* o candados de *OpenMP*.

El alcance de este proyecto incluye la identificación de los recursos compartidos y su protección contra condiciones de carrera, la evidencia de una condición de carrera real mediante la opción de compilación *-DSIN_PROTECCION*, y el analista del rendimiento del sistema con las métricas de *speedup* y eficiencia, con la ley de Amdahl.

Este documento se organiza de la siguiente manera. En la sección de diseño de la solución se describen la arquitectura del sistema, el flujo de ejecución de un *pipeline*, el protocolo cliente-servidor y los estados de las ejecuciones con sus transiciones válidas. En esa misma sección se explica cómo se resolvió cada una de las situaciones de concurrencia planteadas: el límite de agentes, la consulta del estado de las ejecuciones, las bitácoras, las variables de ejecución, los artefactos, la cancelación y el tiempo límite, las tareas con paralelismo de datos, los procesos hijos y el paralelismo anidado. En la sección de baterías de pruebas se presentan los casos de prueba realizados y la evidencia de la condición de carrera. En la sección de métricas se analiza el rendimiento del sistema. Por último, se presentan las conclusiones, las limitaciones del diseño y el trabajo futuro.

2. Diseño de la solución

2.1. Arquitectura

En la figura 1 se muestran los componentes de la solución y su interacción. El flujo es el siguiente: el cliente envía una solicitud vía TCP desde pipectl, un hilo de conexión en el servidor interpreta dicha solicitud y realiza la acción correspondiente. En el caso de ser un submit, se pasa al analizador del archivo enviado, se valida y construye el grafo asociado. La ejecución generada se guarda en el registro y sus trabajos sin dependencias pasan de una vez a la cola de trabajos listos, cola que alimenta a los agentes OpenMP.

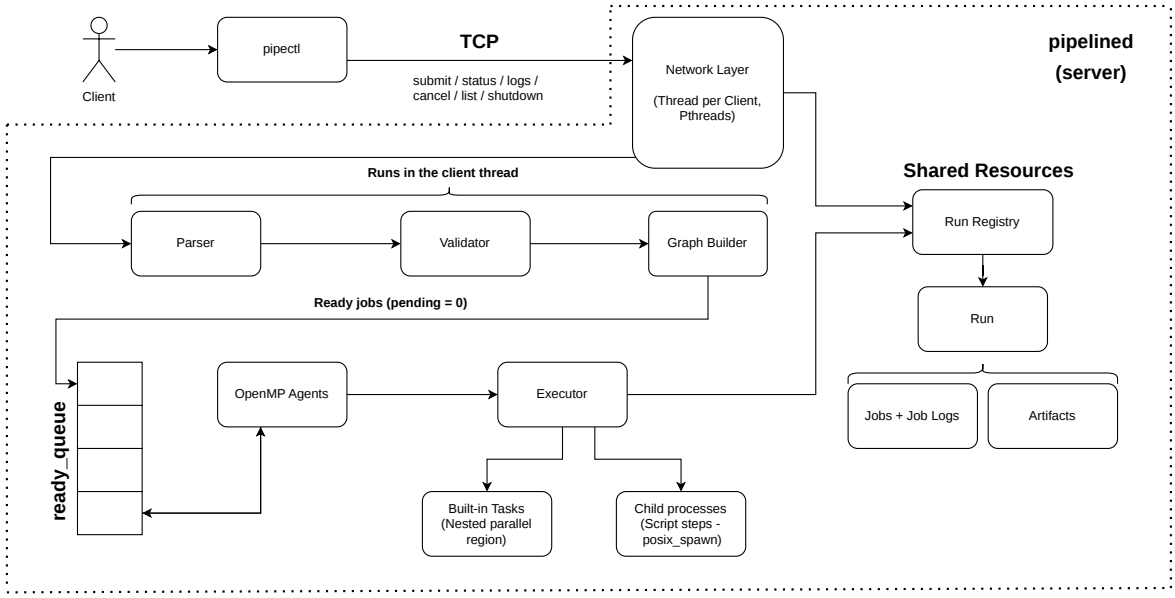


Figura 1: Componentes del servidor y flujo del proceso.

2.2. Flujo de ejecución

1. **Recepción.** El hilo de conexión recibe `SUBMIT <ruta>` y verifica que el archivo indicado exista.

2. **Validación.** El analizador construye la estructura del pipeline y el validador acumula todos los errores encontrados. Si hay errores, se responden todos con su línea y no se crea la ejecución.

3. **Grafo.** Según el enunciado, cada trabajo depende de todos los trabajos de la etapa anterior. En la etapa, `dependsOn` reemplaza la dependencia de la etapa anterior. En un trabajo, agrega dependencias con otros trabajos de su misma etapa. Si el grafo tiene un ciclo, se rechaza indicando el camino del ciclo.

4. **Registro.** Se crea la ejecución, cada trabajo con un contador `pending` igual a su número de predecesores, y se le asigna un identificador.

5. **Inicio.** Se encolan los trabajos con `pending` igual a cero.

6. **Ejecución.** Un agente toma un trabajo, ejecuta sus pasos en orden y fija su estado final.

7. **Avance.** Al terminar, el trabajo decrementa de forma atómica el `pending` de cada sucesor, el sucesor que llega a cero se evalúa y, si su condición se cumple, se encola.

8. **Cierre.** Cuando termina el último trabajo, el estado de la ejecución se calcula a partir de los estados resultantes de sus trabajos.

2.3. Estados y transiciones

Los pasos, los trabajos y las ejecuciones utilizan los mismos siete estados: `queued`, `skipped`, `running`, `canceled`, `succeeded`, `partiallySucceeded` y `failed`. La figura 2 muestra las transiciones válidas para un trabajo.

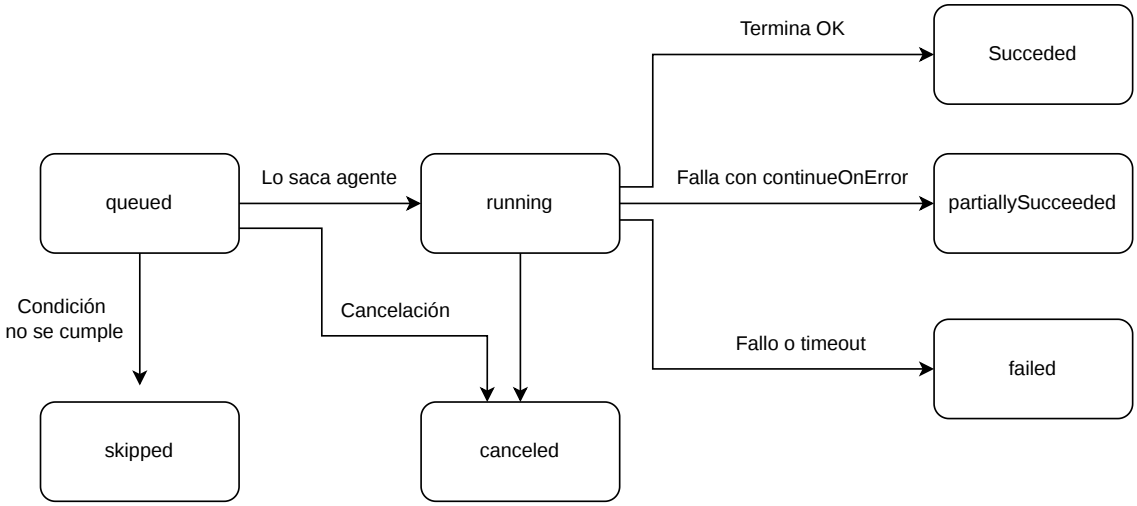


Figura 2: Transiciones de estado de un trabajo.

Una ejecución termina en estado `canceled` si se canceló, `failed` si algún trabajo falló, `partiallySucceeded` si alguno terminó así, y `succeeded` si finalizo correctamente.

Dentro de un trabajo, los pasos se ejecutan en orden. Si un paso falla sin tener `continueOnError`, los pasos siguientes quedan como `skipped`, Si el trabajo se cancela o vence su tiempo límite, quedan como `canceled`.

2.4. Recursos compartidos

La tabla 1 identifica cada recurso compartido, quién lo accede y su mecanismo de protección. Se procuró utilizar OpenMP en todo lo que pertenece a la ejecución, con una excepción aprobada por el docente: la cola de trabajos listos usa un mutex y una variable de condición de POSIX, para que los agentes esperen dormidos mientras la cola está vacía y despierten cuando llega un trabajo. Pthreads se usa además en la capa de red, para los hilos de las conexiones y el mutex del registro de ejecuciones.

Tabla 1: Recursos compartidos.

Recurso	Actor	Mecanismo
Cola de trabajos listos	Agentes (sacan) y quien libera trabajos (agrega)	Mutex y variable de condición de POSIX
Contador `pending` de cada trabajo	Agentes que terminan predecesores	`#pragma omp atomic capture`
Contador `remaining` de la ejecución	Agentes	`#pragma omp atomic capture`
Bandera `cancel_requested`	Hilo de red (escribe), agentes y tareas (leen)	`omp atomic write` / `read`
Estados y tiempos de trabajos y pasos	Agentes (escriben), `status` y `list` (leen)	`omp_lock_t` de la ejecución
Bitácora de cada trabajo	Agente del trabajo (escribe), `logs` (lee)	`omp_lock_t` de la bitácora
Registro de ejecuciones	Hilos de conexión	`pthread_mutex_t` (capa de red)
Variables de salida de un trabajo	Sucesores, después de que el trabajo terminó	Sin bloqueo: se escriben antes de liberar a los sucesores
Carpeta de un artefacto	Trabajos que publican o descargan	Atomicidad de `mkdir`

2.5. Respuestas situaciones planteadas Tabla 5

Límite de agentes

El planificador (scheduler.c) abre una única región con `#pragma omp parallel num_threads(n)`, donde n es el número de agentes que se definió vía parámetro. Se utiliza `omp_set_dynamic(0)` para que OpenMP cree exactamente n hilos.

Estado de las ejecuciones

Los estados y tiempos de ejecución, tanto de los trabajos como de los pasos, están protegidos con un `omp_lock_t`. Los agentes realizan los cambios dentro del bloqueo, status y list leen igual bajo el mismo bloqueo, lo que evita observar un estado en plena actualización.

Bitácoras

Se definió una bitácora por trabajo, protegida con un `omp_lock_t`. Las lecturas con `logs -follow` leen el log mientras el trabajo sigue escribiendo en bitacora. El lock garantiza que cada línea en bitácora se escriba completa antes de ser leída.

Variables de ejecución

Los trabajos inician su ciclo de vida con las variables definidas inicialmente en el pipeline. Posterior a esto se van incorporando las variables de salida de sus predecesores, según la lista de predecesores definida en el grafo. Si dos predecesores definen la misma variable gana la última definida. Al no depender de que predecesor termina primero se garantiza el determinismo.

Artefactos

Los artefactos se manejan como carpetas `artifacts/nombre` dentro del directorio de ejecución. Se utiliza el comando `mkdir` para crear esas carpetas, si la carpeta ya existe falla. Como

`mkdir` es una operación atómica dentro del sistema de archivos, evita tener que usar un bloqueo adicional. Por tanto, si se intentan publicar dos artefactos con el mismo nombre se registrará el primero, el segundo fallará junto con su etapa. Los artefactos se copian con `cp -r` tanto en su publicación como en sus descargas. Si el artefacto no existe al intentar la descarga, el paso asociado falla.

Cancelación y tiempo límite

Cancelar marca la bandera `cancel_requested` de la ejecución. Las tareas en curso están programadas para verificar esta bandera cada 100 ms; al verla, terminan, y su trabajo queda como `canceled`. Los trabajos que estén en la cola salen de ella de forma normal cuando un agente los toma, y este los marca como `canceled` sin ejecutar nada más. Los que todavía esperaban a sus predecesores se marcan como `canceled` cuando estos terminan y les toca ser liberados. Así, no se eliminan trabajos de la cola ni se detienen hilos a la fuerza, y cada trabajo termina con un estado final.

El tiempo límite funciona de forma muy similar: el trabajo calcula su hora límite al iniciar, y cada tarea la revisa en esas mismas verificaciones. En ambos casos, si el paso en curso es un script, el agente que lo vigila envía `SIGKILL` a todo el grupo de procesos del hijo, para que también terminen los procesos que este haya creado. Después lo espera con `waitpid`, de modo que no quede como proceso zombi, y cierra su extremo del pipe.

Tareas con paralelismo de datos

`Histogram@1` reparte el archivo entre los hilos con `#pragma omp parallel for` y combina los conteos con `reduction(+:histograma[:num_bins])`. Como la suma es conmutativa, el resultado es idéntico al serial. `WordCount@1` divide el archivo en un bloque por hilo (*chunk*), ajustando los bordes para no partir palabras. Cada hilo cuenta en su propia lista y después las listas se combinan en serie, en el orden de los bloques. El resultado se ordena por frecuencia, en caso de empate alfabéticamente, así la salida no depende del número de hilos. Para el proyecto se definió que una palabra es una secuencia de letras y dígitos ASCII, sin distinguir mayúsculas de minúsculas.

Procesos hijos

Se decidió utilizar `posix_spawn` por dos razones: Primero, crea el proceso hijo y ejecuta el programa sin correr código del servidor en medio. Con `fork` seguido de `exec`, el hijo es una copia del proceso en la que solo sobrevive el hilo que llamó a `fork`; si en ese instante otro hilo tenía tomado un bloqueo, el hijo lo hereda tomado y nadie lo liberará, por lo que cualquier llamada que lo use antes de `exec` quedaría bloqueada. Segundo, es más liviano, porque en Linux crea el hijo sin copiar la memoria del padre. El pipe hacia el proceso hijo se crea con la bandera `O_CLOEXEC`, que garantiza que los procesos hijos de otros agentes no hereden su extremo de escritura; si lo heredaran y lo mantuvieran abierto, el agente dueño del pipe no recibiría el fin de la salida hasta que esos procesos terminaran.

Paralelismo anidado

Las tareas abren su propia región paralela dentro de cada agente, por lo que se habilitan dos niveles explícitos de paralelismo anidado con `omp_set_max_active_levels(2)`. Para evitar la

sobresuscripción, cada tarea usa como máximo $\lfloor P/n \rfloor$ hilos, donde P es el número de procesadores y n el número de agentes: el valor pedido en el YAML se limita a ese máximo, con un mínimo de un hilo.

3. Baterías de pruebas

El plan de pruebas busca demostrar cuatro características del sistema: la funcionalidad de cada componente, el funcionamiento del servidor completo ante solicitudes válidas e inválidas, la corrección concurrente de los mecanismos de sincronización y el determinismo de los resultados, que no deben depender del orden en que se ejecuten los hilos.

Las pruebas unitarias y de integración del ejecutor son automáticas y se ejecutan con `make test`. Las pruebas del servidor completo se realizan sobre el servidor en ejecución, mediante el cliente `pipectl` y *scripts* que automatizan el envío de solicitudes. La Tabla 2 resume las pruebas realizadas.

Categoría	**Archivo**	**Casos**	**Qué demuestra**
Unitaria	`test_parser.c`	Archivo válido, claves desconocidas, pasos incompletos, nombres duplicados, referencias inexistentes, dependencias cíclicas entre trabajos y entre etapas	Validación del *YAML* y mensajes de error con su número de línea
Unitaria	`test_ready_queue.c`	Orden de salida, cierre de la cola, 10 000 trabajos insertados y extraídos por varios hilos a la vez	Cola de trabajos listos sin pérdida ni duplicación
Unitaria	`test_scheduler.c`	Arranque y apagado con cuatro agentes, rechazo de cero agentes, encolar tras el apagado	Ciclo de vida del scheduler
Unitaria	`test_tasks.c`	`WordCount@1` e `Histogram@1` con uno y cuatro hilos, `Cpu@1` con la misma semilla, entradas inválidas, tarea desconocida, cancelación y tiempo límite	Corrección y determinismo de las tareas integradas
Integración	`test_executor.c`	*Pipeline* completo con artefactos, variables y condiciones; cancelación; tiempo límite. 50 trabajos que alimentan a uno solo, repetido diez veces	Ejecución correcta de un *pipeline* con todos los componentes
Concurrencia	`sin_proteccion.sh`	200 solicitudes *SUBMIT* simultáneas al servidor	Ausencia de condiciones de carrera en el registro de ejecuciones

Tabla 2: Pruebas del proyecto.

Las pruebas se ejecutaron en los ambientes descritos en la Tabla 5 de la sección de métricas.

3.1. Pruebas unitarias

Analizador y validación: La prueba `test_parser.c` procesa un conjunto de archivos *YAML* preparados para cada tipo de error. Además, comprueba que un archivo válido se interprete correctamente, revisando sus etapas, trabajos, dependencias, condiciones y tiempos límite.

Cola de trabajos: La prueba `test_ready_queue.c` verifica que los trabajos salgan en el mismo orden en que entraron y que, una vez cerrada la cola, no se puedan agregar trabajos y los agentes que esperan reciban un aviso de cierre.

Scheduler: La prueba `test_scheduler.c` verifica que el planificador arranque con cuatro agentes y se apague correctamente, que rechace una configuración sin agentes y que no acepte nuevos trabajos una vez iniciado el apagado.

Tareas integradas: La prueba `test_tasks.c` ejecuta `WordCount@1` e `Histogram@1` sobre el mismo archivo con uno y con cuatro hilos, y comprueba que los archivos de salida sean idénticos.

Ejecutor: La prueba `test_executor.c` ejecuta un *pipeline* de dos etapas que combina *scripts*, publicación y descarga de artefactos, variables definidas en tiempo de ejecución, la tarea `WordCount@1`, un paso con `continueOnError` y un trabajo con la condición `failed()`, y verifica el estado final de cada trabajo y de la ejecución. Además, comprueba que la cancelación detenga un trabajo en curso, que el tiempo límite detenga un *script* y haga fallar el trabajo, y que, cuando 50 trabajos alimentan a un único trabajo final, este se ejecute una sola vez.

3.2. Evidencia de la condición de carrera

Para evidenciar una condición de carrera real, la opción de compilación `-DSIN_PROTECCION` elimina el *mutex* que protege la función `run_add` del registro de ejecuciones. Esta función lee el contador de ID's, `next_id`, y escribe el siguiente valor. Sin el *mutex*, dos hilos pueden leer el mismo valor antes de que alguno lo incremente, de modo que dos ejecuciones distintas reciben el mismo ID.

Como la ventana en la que ocurre esta carrera es muy pequeña, en la versión sin protección se agregó una llamada a `sched_yield()` entre la lectura y la escritura del contador, que hace que el hilo ceda el procesador justo en ese punto. Esta llamada no crea la condición de carrera, que existe desde el momento en que se elimina el *mutex*, sino que permite que se manifieste de forma reproducible. En la versión con protección, esta llamada no se compila.

La prueba `sin_proteccion.sh` envía 200 solicitudes *SUBMIT* simultáneas y revisa si algún identificador se repite en las respuestas. La Tabla 3 muestra los resultados.

Versión	**Compilación**	**Resultado**
Sin protección	`make DSIN_PROTECCION`	Condición de carrera detectada: En promedio 8 ID's repetidos
Con protección	`make`	Ningún identificador repetido en 200 envíos simultáneos

Tabla 3: Resultados de la prueba de la condición de carrera.

4. Métricas

El objetivo de esta sección es evaluar el rendimiento del servidor de *pipelines* y analizar qué tanto mejora el tiempo de ejecución al aumentar el número de agentes o de hilos. Para esto se diseñó una batería de *pipelines* de medición que permite estudiar por separado el paralelismo entre trabajos, el efecto de las dependencias y el paralelismo de datos dentro de una tarea. La Tabla 4 resume los escenarios de medición.

Escenario	**Pipeline**	**Se varía**
Ancho	Seis trabajos independientes en una misma etapa	Agentes
Profundo	Los mismos seis trabajos, cada uno dependiente del anterior	Agentes
Paralelismo de datos	Tarea `WordCount@1` sobre un archivo de cientos de megabytes	Hilos de *OpenMP*
Combinado	rellenar	Agentes

Tabla 4: Escenarios de medición.

4.1. Cálculos realizados

Para cada escenario se mide el tiempo de ejecución $T(p)$, donde p es la cantidad de agentes, o de hilos en el escenario de paralelismo de datos. A partir de estos tiempos se calcula el *speedup* con p agentes que con uno solo, y la eficiencia:

$$S(p) = \frac{T(1)}{T(p)} \qquad E(p) = \frac{S(p)}{p}$$

La fracción paralelizable f corresponde a la parte del trabajo que puede repartirse entre los agentes. No se conoce de antemano, sino que se estima a partir del *speedup*, despejándola de la ley de Amdahl:

$$f = \frac{1 - \dfrac{1}{S(p)}}{1 - \dfrac{1}{p}}$$

Con la fracción paralelizable estimada, se calcula el *speedup* que predice la ley de Amdahl para cada valor de p y se compara con el *speedup* medido:

$$S_{\text{Amdahl}}(p) = \frac{1}{(1 - f) + \dfrac{f}{p}}$$

Estos cálculos se realizan para cada uno de los cuatro escenarios de medición. En todos ellos, $T(1)$ corresponde al tiempo de ejecución promedio con un solo agente o un solo hilo.

4.2. Metodología

El tiempo de ejecución de cada *pipeline* lo registra el propio servidor: el ejecutor guarda el instante en que inicia la ejecución y el instante en que termina su último trabajo, y el comando *STATUS* reporta la diferencia en microsegundos. De esta forma, el tiempo medido corresponde únicamente a la ejecución del *pipeline* dentro del servidor, sin incluir el tiempo de comunicación con el cliente.

Las mediciones se automatizan con un *script* que envía cada *pipeline*, espera a que termine y obtiene su duración. Cada punto se mide cinco veces y se reporta la mediana. Las mediciones se realizan en dos ambientes distintos, descritos en la Tabla 5.

Característica	**Ambiente 1**	**Ambiente 2**
Equipo	Computadora personal de uno de los integrantes	Clúster Kabré del CNCA
Procesador	AMD Ryzen 7 5700G with Radeon Graphics	*Intel(R) Xeon(R) Platinum 8160 CPU @ 2.10GHz*
Núcleos / hilos	8 núcleos / 16 hilos	*4*
Memoria RAM	16 GB	*8 GB*
Sistema operativo	Ubuntu 26.04 LTS	*AlmaLinux*
Compilador	GCC 15.2.0	*GCC 11.5.0*

Tabla 5: Ambientes de medición.

4.3. Pipeline ancho

En este escenario, los seis trabajos pertenecen a una misma etapa y no dependen entre sí, por lo que pueden ejecutarse simultáneamente en tantos agentes como haya disponibles. Se midió con 1, 2, 4 y 8 agentes. La Tabla 6 muestra los resultados.

Ambiente	p	$T(p)$ **[segundos]**	$S(p)$	$E(p)$	$S_{\textbf{Amdahl}}(p)$
1	1	18.094	1.00	1.00	1.00
	2	9.023	2.00	1.00	1.91
	4	6.012	3.00	0.75	3.50
	8	3.005	6.01	0.75	6.01
2	1	18.480	1.00	1.00	1.00
	2	9.233	2.00	1.00	1.90
	4	6.184	2.99	0.75	3.44
	8	3.115	5.93	0.74	5.81

Tabla 6: Resultados del *pipeline* ancho. Fracción paralelizable estimada: $f = 0{,}953$ – (ambiente 1), $f = 0.946$ (ambiente 2).

4.4. Pipeline profundo

En este escenario, cada trabajo depende del anterior, de modo que forman una cadena en la que solo un trabajo puede ejecutarse a la vez, sin importar cuántos agentes haya. Se midió con los mismos valores de p que el *pipeline* ancho. La Tabla 7 muestra los resultados.

Ambiente	p	$T(p)$ **[segundos]**	$S(p)$	$E(p)$	$S_{\textbf{Amdahl}}(p)$
1	1	18.042	1.00	1.00	1.00
	2	18.048	1.00	0.50	1.00
	4	18.052	1.00	0.25	1.00
	8	18.023	1.00	0.13	1.00
2	1	18.524	1.00	1.00	1.00
	2	18.354	1.01	0.50	1.01
	4	18.323	1.01	0.25	1.01
	8	18.381	1.01	0.13	1.01

Tabla 7: Resultados del *pipeline* profundo. Fracción paralelizable estimada: $f = 0.001$ (ambiente 1), $f = 0.014$ (ambiente 2).

4.5. Paralelismo de datos

En este escenario se ejecuta un único trabajo con la tarea `WordCount@1` sobre un archivo de aproximadamente 300MB, variando el número de hilos de *OpenMP* con el que la tarea procesa

el archivo. Se utiliza un solo agente, de modo que la única fuente de paralelismo sea el reparto del archivo entre los hilos. La Tabla 8 muestra los resultados.

Ambiente	p	$T(p)$ **[segundos]**	$S(p)$	$E(p)$	$S_{\textbf{Amdahl}}(p)$
1	1	212.052	1.00	1.00	1.00
	2	110.327	1.92	0.96	1.91
	4	60.138	3.53	0.88	3.52
	8	34.971	6.06	0.76	6.06
2	1	270.695	1.00	1.00	1.00
	2	146.910	1.84	0.92	1.79
	4	80.402	3.37	0.84	2.97
	8	80.466	3.36	0.42	4.43

Tabla 8: Resultados del paralelismo de datos. Fracción paralelizable estimada: $f = 0{,}954$ (ambiente 1), $f = 0.885$ (ambiente 2).

4.6. Escenario combinado

En este escenario se envían cinco *pipelines* anchos al mismo tiempo, de modo que sus trabajos compiten por los agentes disponibles. El tiempo $T(p)$ se mide desde que se envían los tres *pipelines* hasta que termina el último. La Tabla 9 muestra los resultados.

Ambiente	p	$T(p)$ **[s]**	$S(p)$	$E(p)$	$S_{\textbf{Amdahl}}(p)$
1	1	90.162	1.00	1.00	1.00
	2	45.088	2.00	1.00	1.98
	4	24.052	3.75	0.94	3.89
	8	12.020	7.50	0.94	7.50
2	1	92.460	1.00	1.00	1.00
	2	46.526	1.99	0.99	1.98
	4	24.577	3.76	0.94	3.85
	8	12.402	7.45	0.93	7.35

Tabla 9: Resultados del escenario combinado. Fracción paralelizable estimada: $f = 0{,}990$ (ambiente 1), $f = 0.987$ (ambiente 2).

5. Conclusiones

El servidor de *pipelines* desarrollado cumple con los requisitos planteados: recibe definiciones en formato *YAML*, rechaza antes de ejecutar aquellas que contienen errores, indicando cada uno con su número de línea, y ejecuta las definiciones válidas de forma concurrente, respetando las dependencias entre trabajos, las condiciones de ejecución y el límite de agentes. Además, permite consultar el estado y las bitácoras de una ejecución mientras esta avanza, cancelarla y apagar el servidor de forma ordenada.

En cuanto a la concurrencia, la identificación explícita de los recursos compartidos permitió elegir para cada uno el mecanismo de sincronización más adecuado. La prueba con `-DSIN_PROTECCION` evidenció que la ausencia de un solo *mutex* basta para que dos ejecuciones reciban el mismo identificador, y mostró también lo difícil que es observar una condición de carrera, ya que su ventana de ocurrencia es de apenas unos nanosegundos.

La mayor parte de la concurrencia se resolvió sin bloqueos porque el grafo de dependencias no cambia durante la ejecución: solo los contadores de predecesores pendientes y las banderas de cancelación cambian, y para ellos basta `omp atomic`. El único bloqueo por ejecución protege los estados y tiempos, que el comando `status` debe leer de forma consistente. El determinismo

se obtuvo de forma análoga, fijando el orden de combinación de los resultados parciales en lugar de dejarlo al orden de terminación de los hilos.

Link al repositorio: https://github.com/SantiagoAsuncionM/Proyecto_Paralela_2